

Matematično-fizikalni praktikum: Strojno učenje

Žan Ambrožič (28231062)

1. marec 2026

1 Naloga

Na spletni učilnici je na voljo material (koda, vzorci) za ločevanje dogodkov Higgsovega bozona od ostalih procesov ozadja. V naboru simuliranih dogodkov je 18 karakteristik (zveznih kinematičnih lastnosti), katerih vsaka posamezno zelo slabo loči 'signal' od ozadja, z uporabo BDT ali (D)NN, pa lahko tu dosežemo zelo dober uspeh. Na predavanjih smo si ogledali glavne aspekte pomembne pri implementaciji ML, kot so uporaba ustreznih spremenljivk (GIGO), učenje in prekomerno učenje (training/overtraining), vrednotenje uspeha metode kot razmerje med učinkovitostjo (efficiency) in čistostjo (precision) vzorca (Receiver Operating Characteristic, ROC). Določi uspešnost obeh metod (in nariši ROC) za nekaj tipičnih konfiguracij BDT in DNN, pri čemer:

- Študiraj vpliv uporabljenih vhodnih spremenljivk - kaj, če vzamemo le nekatere?
- Študiraj BDT in NN in vrednoti uspešnost različnih nastavitev, če spreminjaš nekaj konfiguracijskih parametrov (npr. število perceptronov in plasti nevronske mreže pri DNN in število dreves pri BDT).

2 Uvod

Dandanes je uporaba različnih algoritmov strojnega učenja (Machine Learning, ML) v znanosti že rutinsko opravilo. Poznamo tri osnovne vrste stojnega učenja:

- Nadzorovano učenje (Supervised learning):
 - Klasifikacija (Classification): sortiranje v različne kategorije.
 - Regresija (Regression): modeliranje oz. 'fitanje' napovedi.
- Nenadzorovano učenje (npr. sam najdi kategorije).
- Stimulirano učenje (Artificial Intelligence v ožjem pomenu besede).

V fiziki (in tej nalogi), se tipično ukvarjamo s prvo kategorijo, bodisi za identifikacijo novih pojavov/delcev/... ali pa za ekstrakcijo napovedi (netrivialnih funkcijskih odvisnosti etc).

ML algoritmi imajo prednost pred klasičnim pristopom, da lahko učinkovito razdrobijo kompleksen problem na enostavne elemente in ga ustrezno opišejo:

- pomisli na primer, kako bi bilo težko kar predpostaviti/uganiti pravo analitično funkcijo v več dimenzijah (in je npr. uporaba zlepkov (spline interpolacija) mnogo lažja in boljša).
- Pri izbiri/filtriranju velike količine podatkov z mnogo lastnostmi (npr dogodki pri trkih na LHC) je zelo težko najti količine, ki optimalno ločijo signal od ozadja, upoštevati vse korelacije in najti optimalno kombinacijo le-teh...

Če dodamo malce matematičnega formalizma strojnega učenja: Predpostavi, da imamo na voljo nabor primerov $\mathcal{D} = \{(\mathbf{x}_k, y_k)\}_{k=1..N}$, kjer je $\mathbf{x}_k = (x_k^1, \dots, x_k^M)$ naključno izbrani vektor M lastnosti (karakteristik) in je $y_k = (y_k^1, \dots, y_k^Q)$ vektor Q ciljnih vrednosti, ki so lahko bodisi binarne ali pa realna števila¹. Vrednosti $(\mathbf{x}_k, y_k)$ so

¹...ali pa še kaj, prevedljive na te možnosti, npr. barve...

neodvisne in porazdeljene po neki neznani porazdelitvi $P(\cdot, \cdot)$. Cilj ML metode je določiti (priučiti) funkcijo h : iz Q dimenzij v realna števila, ki minimizira pričakovano vrednost *funkcije izgube* (*expected loss*)

$$\mathcal{L}(h) = \langle L(\mathbf{y}, \mathbf{h}(\mathbf{x})) \rangle = \frac{1}{N} \sum_{k=1}^N L(\mathbf{y}_k, \mathbf{h}(\mathbf{x}_k)).$$

Tu je $L(\cdot, \cdot)$ gladka funkcija, ki opisuje oceno za kvaliteto napovedi, pri čemer so vrednosti $(\mathbf{x}, \mathbf{y})$ neodvisno vzorčene iz nabora $\mathcal{D}$ po porazdelitvi P . Po koncu učenja imamo torej na voljo funkcijo $\mathbf{h}(\mathbf{x})$, ki nam za nek vhodni nabor vrednosti $\hat{\mathbf{x}}$ poda napoved $\hat{\mathbf{y}} = \mathbf{h}(\hat{\mathbf{x}})$, ki ustrezno kategorizira ta nabor vrednosti.

Funkcije $\mathbf{h}$ so v praksi sestavljene iz (množice) preprostih funkcij z (neka) prostimi parametri, kar na koncu seveda pomeni velik skupni nabor neznanih parametrov in zahteven postopek minimizacije funkcije izgube.

Osnovni gradnik odločitvenih dreves je tako kar stopničasta funkcija $H(x_i - t_i) = 0, 1$, ki je enaka ena za $x_i > t_i$ in nič drugače in kjer je x_i ena izmed karakteristik in t_i neznani parameter. Iz skupine takšnih funkcij, ki predstavljajo binarne odločitve lahko skonstruiramo končno uteženo funkcijo

$$\mathbf{h}(\mathbf{x}) = \sum_{i=1}^J \mathbf{a}_i H(x_i - t_i),$$

kjer so $\mathbf{a}_i$ vektorji neznanih uteži. Tako t_i kot $\mathbf{b}_i$, lahko določimo v procesu učenja. Nadgradnjo predstavljajo nato *pospešena* odločitvena drevesa (BDT), kjer nadomestimo napoved enega drevesa z uteženo množico le-teh, tipično dobljeno v ustreznih iterativnih postopkih (npr. AdaBoost, Gradient Boost ipd.).

Pri nevronske mrežah je osnovni gradnik t.i. *perceptron*, ki ga opisuje preprosta funkcija

$$h_{w,b}(\mathbf{X}) = \theta(\mathbf{w}^T \cdot \mathbf{X} + b),$$

kjer je $\mathbf{X}$ nabor vhodnih vrednosti, $\mathbf{w}$ vektor vrednosti uteži, s katerimi tvorimo uteženo vsoto ter b dodatni konstanti premik (bias). Funkcija θ je preprosta gladka funkcija (npr. arctan), ki lahko vpelje nelinearnost v odzivu perceptrona. Nevronska mreža je nato sestavljena iz (poljubne) topologije takšnih perceptronov, ki na začetku sprejme karakteristiko dogodka $\mathbf{x}$ v končni fazi rezultirajo v napovedi $\hat{\mathbf{y}}$, ki mora seveda biti čim bližje ciljni vrednosti $\mathbf{y}$. Z uporabo ustrezne funkcije izgube (npr. MSE: $\mathcal{L}(h) = \langle \|\mathbf{y} - \hat{\mathbf{y}}\|^2 \rangle$), se problem znova prevede na minimizacijo, kjer iščemo optimalne vrednosti (velikega) nabora uteži $\mathbf{w}_i$ ter b_i za vse perceptrone v mreži. Globoke nevronske mreže (DNN) niso nič drugega, kot velike nevronske mreže ali skupine le-teh.

Že namizni računalniki so dovolj močni za osnovne računske naloge, obstajajo pa tudi že zelo uporabniku prijazni vmesniki v jeziku Python, na primer:

- Scikit-Learn (*scikit-learn.org*): odprtokodni paket za strojno učenje,
- TensorFlow (*tensorflow.org*): odprtokodni Google-ov sistem za ML, s poudarkom na globokih nevronske mrežah (Deep Neural Networks, DNN) z uporabo vmesnika Keras. Prilagojen za delo na GPU in TPU.
- Catboost: (*Catboost.ai*) : odprtokodna knjižnica za uporabo pospešenih odločitvenih dreves (Boosted Decision Trees, BDT). Prilagojena za delo na GPU.

3 Set podatkov

Set podatkov vsebuje okoli 11 milijonov podatkov simuliranih trkov z 28 spremenljivkami. Set razdelimo na tri podsete: za trening uporabimo 80 % podatkov, za evalvacijo 10 % in za test preostalih 10 %.

3.1 Spremenljivke

Ime spremenljivke	Opis
lepton pT	Transverzalni impulz leptonov (p_T^ℓ).
lepton eta	Pseudorapidnost leptonov (η^ℓ).
lepton phi	Azimutni kot leptonov (ϕ^ℓ).
missing energy	Manjkajoča transversalna energija (MET).
missing energy phi	Azimutni kot manjkajoče transversalne energije.
jet.i pt	Transverzalni impulz i -tega curka.
jet.i eta	Pseudorapidnost i -tega curka.
jet.i phi	Azimutni kot i -tega curka.
jet.i b-tag	Prisotnost b -kvarka v i -tem curku.
m_jj	Invariantna masa sistema dveh curkov (m_{jj}).
m_jjj	Invariantna masa sistema treh curkov (m_{jjj}).
m_lv	Invariantna masa sistema lepton + nevtrino (ocenjena iz MET).
m_jlv	Invariantna masa sistema curek + lepton + nevtrino.
m_bb	Invariantna masa parab-označenih curkov.
m_wbb	Invariantna masa sistema $W + b + b$.
m_wwbb	Invariantna masa sistema $W + W + b + b$.

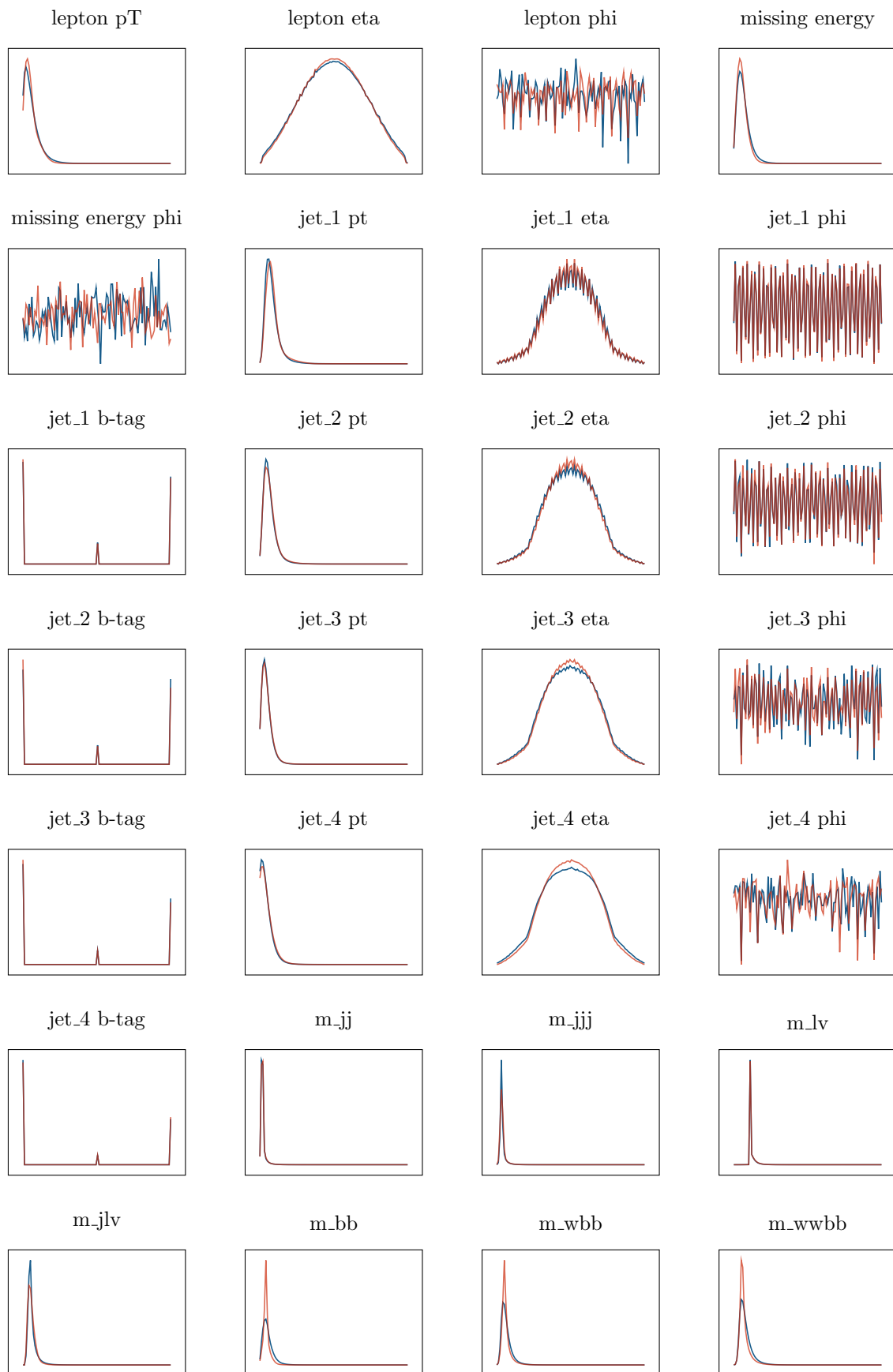
Tabela 1: Spremenljivke podatkovnega seta. V zgornjem delu so neposredno merjene spremenljivke, v spodnjem pa izpeljane. Kjer uporabljamo indeks i , le ta teče od 1 do 4.

Podatke za trening normaliziramo med 0 in 1 ter enako reskaliramo še evalvacijske in testne podatke.

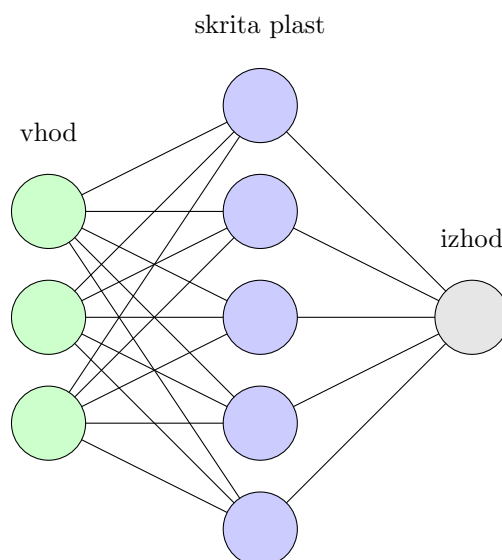
4 Nevronska mreža

Nevronska mreža je splet nevronov, ki na eni strani sprejme podatke, na drugi strani pa vrne napoved. Nevroni v vmesnih plasteh se praviloma aktivirajo v smeri od vhoda proti izhodu. Vsaka povezava na skici 1 predstavlja en parameter – utež povezave. Vsak nevron (krog na skici) pa kot vhod prejme vsoto obteženih vrednosti povezanih nevronov prejšnje plasti, sam pa ji lahko prišteje še dodaten konstantni člen. Kaj potem dobimo na izhodu posameznega nevrona, določa aktivacijska funkcija, ki sprejme dobljeno vsoto. Število parametrov, ki jih prilagajamo, je tako enako skupnemu številu vseh povezav in nevronov.

Ko inicializiramo novo mrežo, so vsi parametri naključni, nato pa jih glede na uspešnost napovedi prilagajamo. Uspešnost napovedi merimo s funkcijo izgube, prilagajanje vrednosti (optimizacijo) pa izvaja optimizator (običajno Adam) – le ta uravnava, kako velike korake delamo pri popravkih parametrov (hitrost učenja) in kako nerabljene povezave čez čas oslabijo.



Graf 1: Distribucije v setu podatkov Higgs po parametrih, modra prikazuje ozadje, rdeča pa signal, na pogled sta praktično neločljiva, zato detekcija ni trivialna. Vzorčenje je na 100 enakomernih intervalov znotraj območja posameznega parametra.



Skica 1: Preprosta shema mreže.

4.1 Prvi test NN

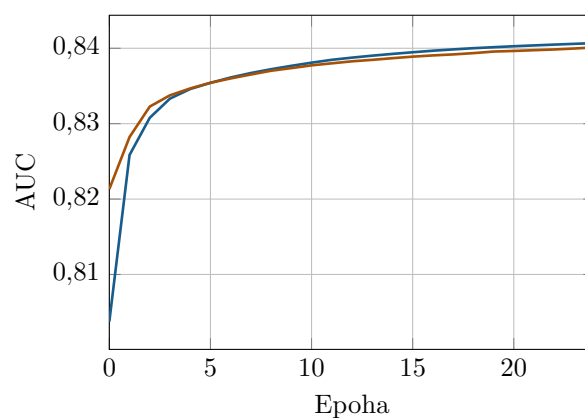
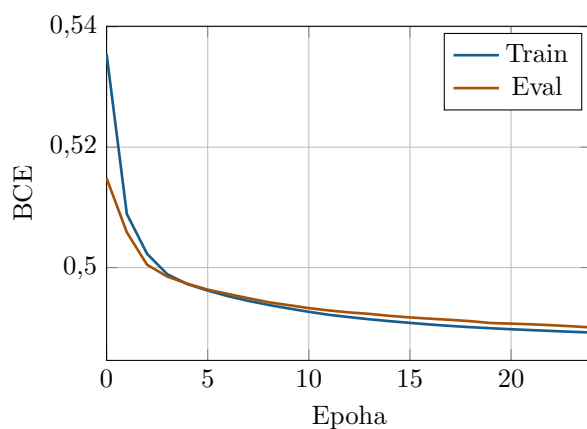
Predlagan osnovni model za ta podatkovni set je model z dvema 50-nevronska plastema (ki so vsi medsebojno prepleteni, aktivacija ReLU) in linearnim izhodom glede na nevrone v zadnji plasti (aktivacija: sigmoida). Za optimizator uporabimo Adam, za funkcijo izgube pa BCE. Za začetek model treniramo s po 1000 vzorci naenkrat 25 epoh.

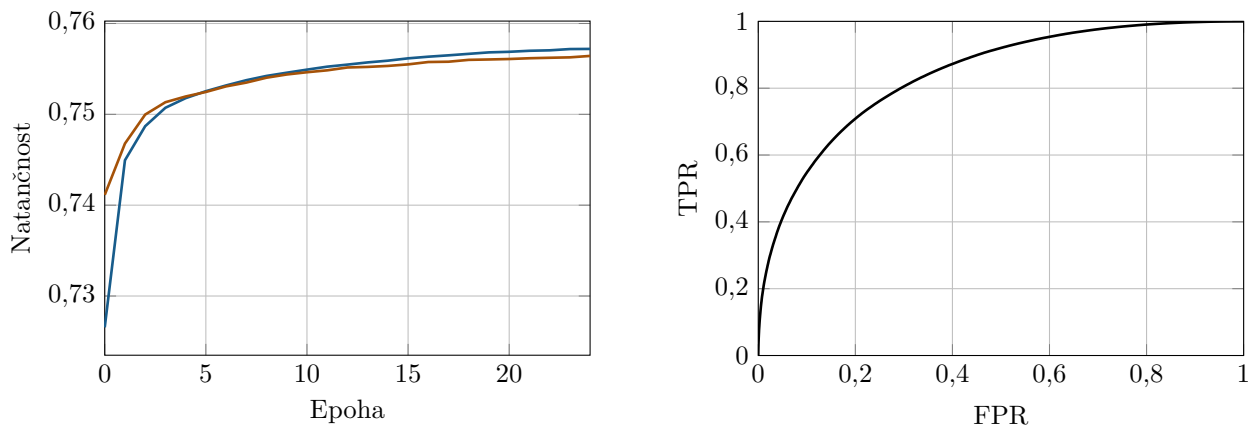
Vsak izhod modela lahko postavimo v eno kategorijo:

- TP: pozitivna napoved za signal,
- FP: pozitivna napoved za ozadje,
- FN: negativna napoved za signal,
- TN: negativna napoved za ozadje.

S tem lahko definiramo eno od standardnih metrik – natančnost, ki nam pove celoten delež pravilnih napovedi $(TP+TN) / (TP+FP+FN+TN)$. Za risanje ROC krivulje pa moramo uvesti še deleža pravilno pozitivnih in lažno pozitivnih vzorcev:

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}$$





Graf 2: Vrednosti funkcije izgube, AUC in natančnosti tekom treninga za trening in validacijski podset ter končna ROC krivulja.

Ker je naš set podatkov (in tudi vsi izbrani podseti) uravnovešen, je meja za pozitivno zaznavo 0,5 zelo dober približek in mu ni potrebno posebej posvečati pozornosti.

4.2 Odvečni vhodni parametri

Na stremeniranem modelu lahko preverimo, kateri vhodni podatki vplivajo na točnost rezultatov. To storimo tako, da v testnem podsetu spreminjamo po en parameter naenkrat in opazujemo ali AUC pade. Če je sprememba majhna, parameter ne spremeni napovedi modela, kar pomeni, da ga model praktično ne upošteva, zato ga brez škode na natančnosti lahko ignoriramo, s čimer pa pospešimo izračune (saj zmanjšamo število parametrov v mreži).

Parameter	Padec AUC	Odveč
lepton pT	0,045	
lepton eta	0,024	
lepton phi	-0,000004	X
missing energy	0,025	
missing energy phi	-0,000008	X
jet_1 pt	0,051	
jet_1 eta	0,015	
jet_1 phi	0,000027	X
jet_1 b-tag	0,0083	
jet_2 pt	0,020	
jet_2 eta	0,0075	
jet_2 phi	-0,000021	X
jet_2 b-tag	0,0045	
jet_3 pt	0,0073	
jet_3 eta	0,0030	
jet_3 phi	-0,000002	X
jet_3 b-tag	0,0039	
jet_4 pt	0,0047	
jet_4 eta	0,0017	
jet_4 phi	-0,00001	X
jet_4 b-tag	0,0043	
m_jj	0,029	
m_jjj	0,049	
m_lv	0,011	
m_jlv	0,054	
m_bb	0,11	
m_wbb	0,12	
m_wbbb	0,096	

Tabela 2: Vpliv spremembe posameznega parametra v testnem podsetu na rezultat (AUC). 1000 vzorcev naenkrat, 20 epoh. Parametri, katerih sprememba ni znatno znižala AUC, so odveč.

Hitro opazimo, da phi parametrov ne potrebujemo, kar je tudi fizikalno smiselno, saj so dogodki v detektorju rotacijsko invariantni glede na smer vpadlih delcev. Enako bi lahko sklepali že na podlagi njihve distribucije, ki je uniformna. S tem v prvem sloju modela zmanjšamo število parametrov s 1450 na 1150, ne da bi karkoli izgubili na natančnosti napovedi.

4.3 Vpliv konfiguracijskih parametrov

Za bazo vzamemo zgoraj opisan model brez odvečnih parametrov, ki ga treniramo 20 epoh s 1000 vzorci naenkrat. Po enega naenkrat spreminjamo in kot metriko uspešnosti uporabimo AUC na testnem podsetu. AUC baze je 0,838. Vseh možnosti je seveda preveč, da bi jih preizkusili vse naenkrat, pa tudi naša trenutna izbira načina treninga je dokaj standardna, saj se izbrano v največ primerih izkaže kot najučinkovitejše in je zato dobra začetna izbira. Spreminjati se splača predvsem velikost mreže in trajanje treninga.

Sprememba parametra	Sprememba AUC
Število nevronov na skrito plast: 40	−0,0016
Število nevronov na skrito plast: 100	0,012
Le ena skrita plast	−0,019
Dodatna skrita plast 50 nevronov	0,007
Število vzorcev naenkrat: 100	0,0015
Število vzorcev naenkrat: 5000	−0,003
Aktivacija skritih plasti: sigmoida	−0,003
Aktivacija izhoda: ReLU	−0,016
Optimizator: SGD	−0,019
Funkcija izgube: MSE	−0,04
Le merjeni podatki	−0,12
Le izpeljani podatki	−0,06

Tabela 3: Vpliv sprememb posameznih parametrov modela oz. treninga na AUC na testnem podsetu podatkov.

Očitno bi nekoliko večja mreža lahko dosegla boljše rezultate. Velja omeniti, da manjše število vzorcev naenkrat privarčuje pri spominu, vendar je časovno potratnejše. Kot predpostavljeno, sta optimizator Adam in funkcija izgube BCE že precej optimalni možnosti. Ostale spremembe (razen aktivacije izhoda) ne vplivajo znatno, saj pri ponovnem treningu z istimi parametri in na istem podsetu dobimo odstopanja tudi do nekaj desetink odstotka. Z uporabo 100 nevronov na skrito plast in 25 epoh dobimo AUC 0,851, verjetno pa se da še za kakšen odstotek izboljšati rezultat. Z uporabo Youdenovega indeksa (maksimum TPR – FPR) določimo optimalno mejo za ločevanje med signalom in ozadjem, ki je 0,53, kar je res blizu 0,5, je pa tudi enako deležu negativnih vzorcev v setu podatkov. S tem modelom sedaj res dobimo optimalne napovedi.

5 Pospešena odločitvena drevesa

Odločitvena drevesa so struktura, ki vsak vhodni podatek usmeri v eno od vej (običajno **true** in **false**) in na koncu do lista, ki predstavlja naš izhod. Namesto nastavljanja uteži nevronov, pri tem pristopu nastavljamo klasifikacijske pogoje na razvejitvah, pri čemer želimo to storiti kar se da učinkovito – tj. v višjih slojih želimo razdeliti širše možnosti, medtem ko v nižjih gledamo manjše razlike. Prednosti odločitvenih dreves so tudi pregledna struktura in neobčutljivost na skalo podatkov (predhodna normalizacija ni potrebna), običajno imajo manj parametrov od nevronskih mrež, vendar je zato treba paziti, da jih ne pretreniramo. Pospešena odločitvena drevesa so struktura več dreves, na podlagi izhoda vsakega od njih pa nato izračunamo končen izhod. Pri treningu na podatkih algoritem upošteva napake vseh dreves pri prilagajanju vsakega posameznega. Glede na izbiro konfiguracijskih parametrov (maksimalna globina, število dreves, ipd.) algoritem sam sestavi odločitvena drevesa in določi uteži, ki se na koncu uporabljajo za izračun izhoda.

5.1 Prvi test BDT

Za začetek preizkusimo naslednje nastavitve (brez vhodnih parametrov, ki smo jih prepoznali kot odvečne): 300 iteracij na drevesih z globino do 5, za funkcijo izgube uporabimo **Logloss**, za evalvacijsko metriko AUC, stopnjo učenja pa nastavimo na 0,1. Na testnih podatkih dobimo AUC 0,810.

5.2 Vpliv konfiguracijskih parametrov

Ponovno spreminjamo po en konfiguracijski parameter naenkrat in opazujemo, kako se spreminja uspešnost modela.

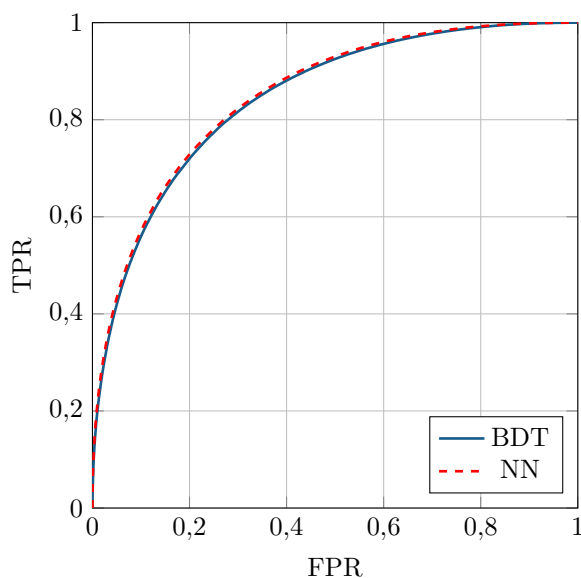
Sprememba parametra	Sprememba AUC
Število iteracij: 500	0,007
Število iteracij: 200	−0,006
Največja globina: 10	0,021
Največja globina: 3	−0,013
Stopnja učenja 0,2	0,009
Stopnja učenja 0,05	−0,009
Funkcija izgube: CrossEntropy	0,0001
Metrika: CrossEntropy	0,000 05
Metrika: Logloss	0,000 20
Le merjeni podatki	−0,10
Le izpeljani podatki	−0,028

Tabela 4: Vpliv sprememb posameznih parametrov BDT oz. treninga na AUC na testnem podsetu podatkov.

Z nadaljnim poskušanjem izboljševanja rezultatov dosežemo lahko tudi AUC 0,846 (in verjetno se da rezultat še nekoliko izboljšati). Spremenjene nastavitve so: največja globina 15, stopnja učenja 0,2 in 500 iteracij. Pri tej globini sicer AUC 0,8 dosežemo že po nekaj deset iteracijah s stopnjo učenja 0,2, nadaljne iteracije pa nas nato pripeljejo do maksimalne natančnosti, preden pridemo v območje pretreniranja.

5.3 Primerjava

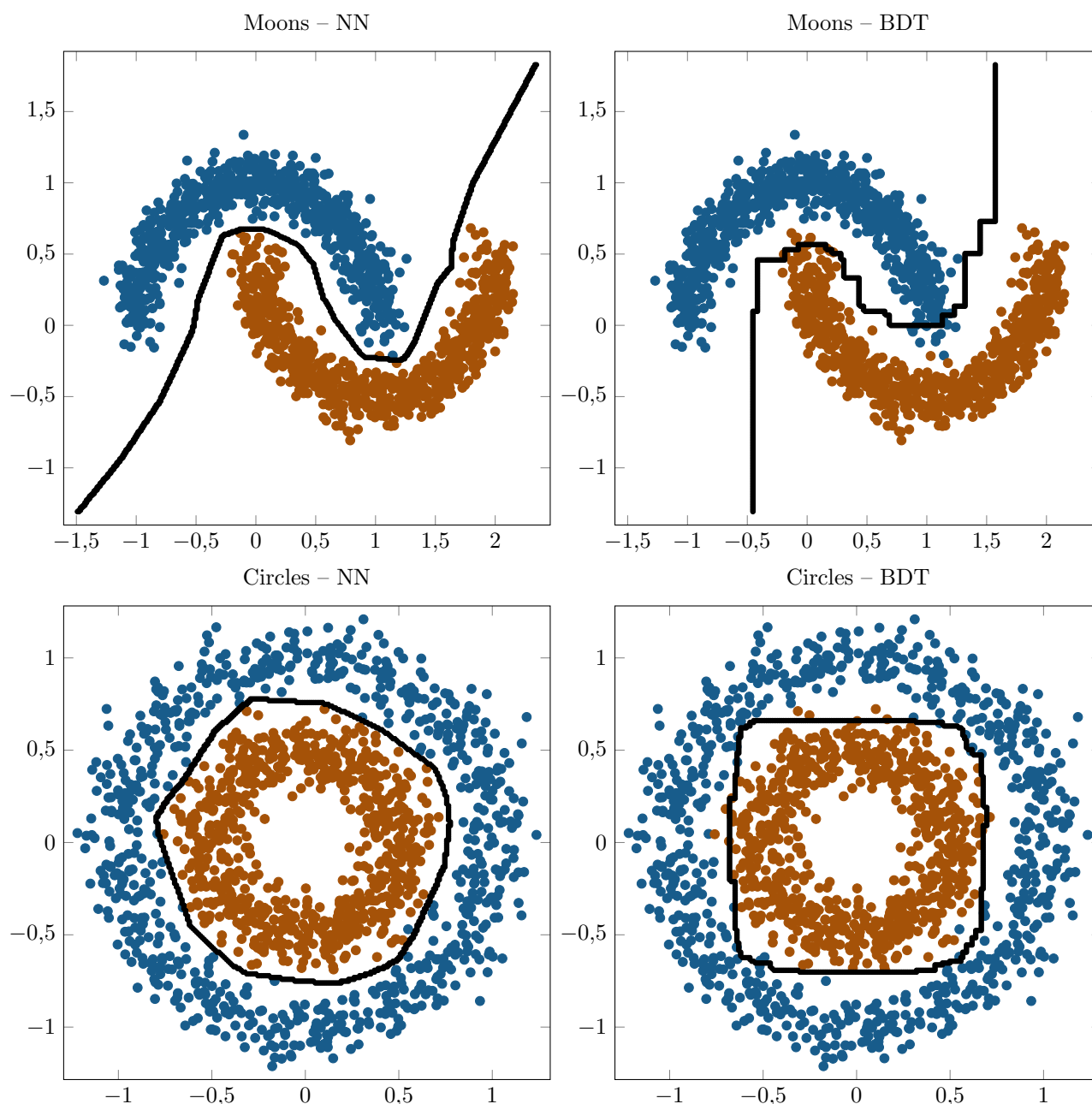
Zadnjo izvedbo modela (z AUC 0,846) primerjamo z najboljšim preizkušenim modelom nevronske mreže (100 nevronov na skrito plast, 25 epoh).



Graf 3: ROC krivulji za najboljša modela (BDT in NN). Obliki sta praktično enaki, zaradi podobnega rezultata (AUC) pa se tudi prekrivata – sta enako oddaljena od diagonale.

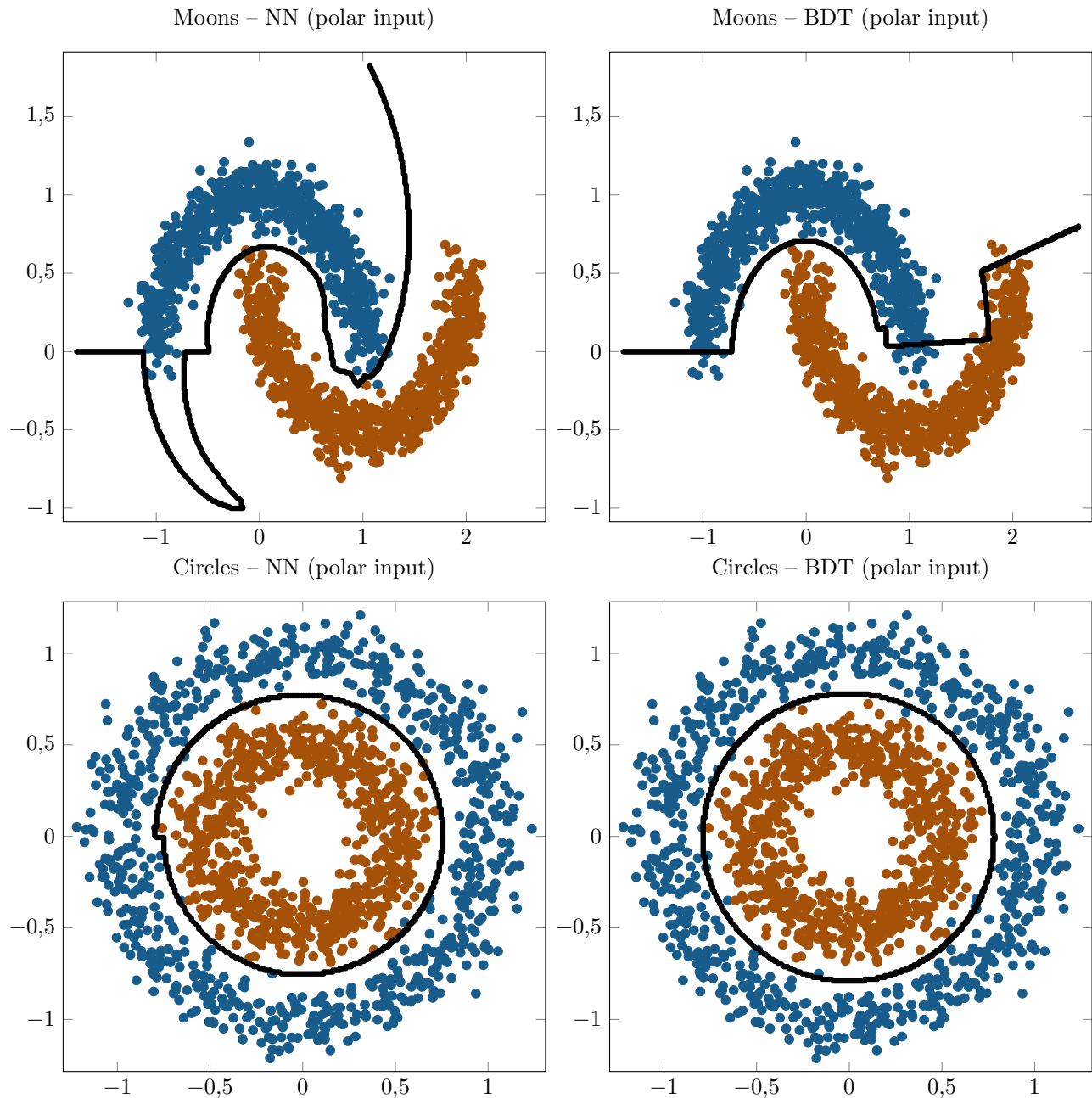
6 Dodatna naloga

Pri dodatni nalogi bomo imeli opravka le z dvoparametričnim setom podatkov, zato je smiselno spremeniti arhitekturo nevronske mreže – več skritih plasti s po manj nevroni – na ta način lažje dobimo tudi razvoje v vrste, saj uporabljamo le linearne operacije (z izjemo sigmoide) – še boljši bi bil prehod na polarne koordinate, vendar bomo poskusili kar s kartezičnimi. Zopet podatke razdelimo na podsete za trening, validacijo in test (vzamemo jih 1500, 60 % za trening, po 20 % za validacijo in test). Tako nevronska mreža kot tudi odločitvena drevesa dosežejo zelo visoke ($> 97\%$) natančnosti – odvisno od semena naključnih podatkov. Zašumljenost setov je bila nastavljena na 0,1, zato se le redko kakšni pozitivni in negativni prekrivajo, kadar pa se, pa zaradi tega ne dobimo popolnoma pravilne napovedi.



Graf 4: Nevronske mreže in pospešenih odločitvenih dreves na Moons in Circles setih podatkov – črna črta prikazuje ločnico med pozitivnimi in negativnimi napovedmi. Prikazani so celotni seti (ne le testni podatki).

Kljub podobni natančnosti pa opazimo med mejami precej razlik, kar je povezano z uporabo kartezičnih koordinat – odločitvena drevesa pri razvejanju vedno uporabljajo le meje x ali y koordinate, zato je pri njihovih napovedih meja bolj "kockasta", medtem ko je pri nevronske mreže gladka.



Graf 5: Nevronske mreže in pospešenih odločitvenih dreves na Moons in Circles setih podatkov v polarnih koordinatah – črna črta prikazuje ločnico med pozitivnimi in negativnimi napovedmi. Prikazani so celotni seti (ne le testni podatki).

V polarnih koordinatah postane ločnica pri krogih veliko jasnejša, zanimive spremembe pa opazimo tudi pri lunicah. Se pa natančnosti ne spremenijo znatno, saj kot že prej omenjeno oba seta delujeta dovolj dobro že v kartezičnih, da od 100 % ločujejo le prekrivajoči se podatki, vsekakor pa smo ponovno demonstrirali, da ni vseeno, v kakšni obliki vhodne podatke dobi model.

7 Zaključek

Pri nalogi smo še dodatno poglobili znanje o strojnem učenju s preizkusi preprostih nevronske mreže in pospešenih odločitvenih dreves. Na standardnem Higgs setu podatkov smo preverjali učinkovitost NN in BDT ter preverjali vpliv različnih vhodnih podatkov in konfiguracijskih parametrov. Že z dokaj enostavnim modelom smo dosegli AUC 0,85, kar je že precej blizu zgornje meje za ta set. Za konec smo preizkusili še vgrajena standardna seta Moons in Circles, kjer smo zaradi 2D podatkov lahko opazovali, kako izgleda meja med pozitivnimi in negativnimi vzorci glede na model in vhodne podatke.